
Integration Manual

for MPC560XB CAN Driver

Document Number: IM13CANASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
----------------	-------	------

Chapter 1 Revision History

Chapter 2 Introduction

2.1	Supported Derivatives.....	7
2.2	Overview.....	8
2.3	About this Manual.....	8
2.4	Acronyms and Definitions.....	8
2.5	Reference List.....	9
2.6	Symbolic Names DISCLAIMER.....	9

Chapter 3 Building the Driver

3.1	Build Options.....	11
3.1.1	GHS Compiler/Linker/Assembler Options.....	11
3.1.2	DIAB Compiler/Linker/Assembler Options.....	13
3.1.3	CW Compiler/Linker/Assembler Options.....	14
3.1.4	HT Compiler/Linker/Assembler Options.....	16
3.2	Files Required for the Compilation.....	18
3.3	Setting up the Plugins.....	19

Chapter 4 Function Calls to Module

4.1	Function Calls during Start-up.....	24
4.2	Function Calls during Shutdown.....	24
4.3	Function Calls during Wake-up.....	26

Chapter 5 Module Requirements

5.1	Exclusive Areas.....	27
5.2	Peripheral Hardware Requirements.....	29

Section number	Title	Page
5.3	ISR to Configure within OS - Dependencies	29
5.3.1	The following ISR's are used by the CAN driver.....	29
5.3.2	Macros for Interrupts.....	31
5.4	ISR Macro.....	34
5.5	Other AUTOSAR Modules - Dependencies.....	35

Chapter 6 Main API Requirements

6.1	Main Functions Calls within BSW Scheduler.....	37
6.2	API Requirements.....	38
6.3	Calls to Notification Functions, Callbacks, Callouts.....	38

Chapter 7 Memory Allocation

7.1	Sections to be defined in MemMap.h.....	41
7.2	Linker Command File.....	43

Chapter 8 Configuration Parameters

8.1	Configuration Parameters.....	45
-----	-------------------------------	----

Chapter 9 Integration Steps

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	15/02/2012	Navnath	Updated for MPC560xB BETA 0.9.0 Release
1.1	07/05/2013	Navnath	Updated for MPC560xB RTM 1.0.0 Release
1.2	06/06/2014	Linh Nguyen Van	Updated for MPC560xB RTM 1.0.1 Release



Chapter 2

Introduction

This integration manual describes the integration requirements for CAN Driver for MPC560XB microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor:

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003 CAN Driver Software Specification Document.	4.0.0 Rev 0003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
8	Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
9	Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

2.6 Symbolic Names DISCLAIMER

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing #ifdefs arguments).

Chapter 3

Building the Driver

This section describes the source files and various compilers, linker options used for building the Autosar CAN driver for Freescale Semiconductor MPC560XB. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

The CAN driver files are compiled using

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

Note: The TS_T2D13M10I1R0 plugin name is composed as follow:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and Derivative_Id = 13 identifies the MPC560XB)

3.1.1 GHS Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-3. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 CW Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch

Table continues on the next page...

Table 3-7. Compiler Options (continued)

Option	Description
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)

Table continues on the next page...

Table 3-8. Assembler Options (continued)

Option	Description
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-9. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE

Table continues on the next page...

Table 3-10. Compiler Options (continued)

Option	Description
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like --noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,' Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script file sprig.link. The -c is a synonym of the -T option

3.2 Files Required for the Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the CAN driver for MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same `AR_RELEASE_MAJOR_VERSION` and `AR_RELEASE_MINOR_VERSION`, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

CAN Files

- `..\Can_TS_T2D13M10I1R0\include\Can.h`
- `..\Can_TS_T2D13M10I1R0\include\Can_LLD.h`
- `..\Can_TS_T2D13M10I1R0\include\Can_FlexCan_LLD.h`
- `..\Can_TS_T2D13M10I1R0\include\Reg_eSys_FlexCan.h`
- `..\Can_TS_T2D13M10I1R0\src\Can.c`
- `..\Can_TS_T2D13M10I1R0\src\Can_LLD.c`
- `..\Can_TS_T2D13M10I1R0\src\Can_Irq.c`
- `..\Can_TS_T2D13M10I1R0\src\Can_FlexCan_LLD.c`

CAN Generated Files

- `Can_PBcfg.c` (For PB Variant) - This file should be generated by the user using a configuration tool for compilation.
- `Can_Cfg.c` (For PC Variant) - This file should be generated by the user using a configuration tool for compilation.
- `Can_Cfg.h` (For PC and PB Variants) - This file should be generated by the user using a configuration tool for compilation.

Files from Base common folder

- `..\Base_TS_T2D13M10I1R0\include\Compiler.h`
- `..\Base_TS_T2D13M10I1R0\include\Compiler_Cfg.h`
- `..\Base_TS_T2D13M10I1R0\include\ComStack_Types.h`
- `..\Base_TS_T2D13M10I1R0\include\MemMap.h`
- `..\Base_TS_T2D13M10I1R0\include\Mcal.h`
- `..\Base_TS_T2D13M10I1R0\include\Platform_Types.h`
- `..\Base_TS_T2D13M10I1R0\include\Std_Types.h`
- `..\Base_TS_T2D13M10I1R0\include\Reg_eSys.h`
- `..\Base_TS_T2D13M10I1R0\include\Soc_Ips.h`
- `..\Base_TS_T2D13M10I1R0\include\Reg_Macros.h`

- ..\Base_TS_T2D13M10I1R0\include\Cer.h
- ..\Base_TS_T2D13M10I1R0\include\Can_GeneralTypes.h

Files from CanIf folder:

- ..\CanIf_TS_T2D13M10I1R0\include\CanIf_Cbk.h
- ..\CanIf_TS_T2D13M10I1R0\include\ComStack_Types.h

Files from Det folder:

- ..\Det_TS_T2D13M10I1R0\src\Det.c
- ..\Det_TS_T2D13M10I1R0\include\Det.h

Files from Rte folder:

- ..\Rte_TS_T2D13M10I1R0\src\Schm_Can.c
- ..\Rte_TS_T2D13M10I1R0\include\Schm_Can.h

Files from EcuM folder:

- ..\EcuM_TS_T2D13M10I1R0\src\EcuM.c
- ..\EcuM_TS_T2D13M10I1R0\include\EcuM.h
- ..\EcuM_TS_T2D13M10I1R0\include\EcuM_Cbk.h

3.3 Setting up the Plugins

The CAN driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.2.1 (b140128-1223) or later.)

Location of various files inside the CAN module folder:

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format:
 - ..\Can_TS_T2D13M10I1R0\config\Can.xdm
 - ..\CanIf_TS_T2D13M10I1R0\config\CanIf.xdm
 - ..\EcuM_TS_T2D13M10I1R0\config\EcuM.xdm
 - ..\Base_TS_T2D13M10I1R0\config\Base.xdm
 - ..\Resource_TS_T2D13M10I1R0\config\Resource.xdm
 - ..\Mcu_TS_T2D13M10I1R0\config\Mcu.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - ..\Can_TS_T2D13M10I1R0\autosar\Can.epd
 - ..\CanIf_TS_T2D13M10I1R0\autosar\CanIf.epd
 - ..\EcuM_TS_T2D13M10I1R0\autosar\EcuM.epd
 - ..\Mcu_TS_T2D13M10I1R0\autosar\Mcu.epd
- Code Generation Templates for Pre-Compile time configuration parameters:
 - ..\Can_TS_T2D13M10I1R0\generate_PC\include\Can_Cfg.h

- ..\Can_TS_T2D13M10I1R0\generate_PC\include\Can_Cfg.c
- ..\Can_TS_T2D13M10I1R0\generate_PC\Can_SourceClock.m
- ..\Can_TS_T2D13M10I1R0\generate_PC\Can_VersionCheck_Inc.m
- ..\Can_TS_T2D13M10I1R0\generate_PC\Can_VersionCheck_Src.m
- ..\Can_TS_T2D13M10I1R0\generate_PC\Can_NotifyCheck_Src.m
- Code Generation Templates for Post-Build time configuration parameters:
 - ..\Can_TS_T2D13M10I1R0\generate_PB\include\Can_Cfg.h
 - ..\Can_TS_T2D13M10I1R0\generate_PB\src\Can_PBcfg.c
 - ..\Can_TS_T2D13M10I1R0\generate_PB\Can_SourceClock.m
 - ..\Can_TS_T2D13M10I1R0\generate_PB\Can_VersionCheck_Src_PB.m
 - ..\Can_TS_T2D13M10I1R0\generate_PB\Can_NotifyCheck_Src_PB.m

Steps to generate the configuration:

1. Copy the module folders Can_TS_T2D13M10I1R0, CanIf_TS_T2D13M10I1R0, Base_TS_T2D13M10I1R0, Resource_TS_T2D13M10I1R0, EcuM_TS_T2D13M10I1R0, Mcu_TS_T2D13M10I1R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

Dependencies

- **MCU** is required to use System Clock when clock source is used as Peripheral clock source to generate CAN Segment values.
- **RESOURCE** is required to select processor derivative. Current Can driver has support for the following derivatives, everyone having attached a Resource file:
mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64
mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100
mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100
mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144
mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64
mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208
mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100
mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176
mpc5607bxmg_mapbga208.
- **CANIF** is required for reporting status of some events.

- **ECUM** is required for selecting the reference to the wakeup source for every Can controller.
- **DET** is required for signaling the development error detection (parameters out of range, null pointers, etc).

Chapter 4

Function Calls to Module

The CAN module shall be initialized by Can_Init(Can_configuration) service call during the start-up.

Note

The settings for pins Rx/Tx required for CAN bus communication are not related to Can driver or plugin configuration.

Note

After Can driver initialization the messages will not be handled for specific controller. API service Can_SetController_Mode(can_controller, CAN_T_START) shall be used for setting the CAN controller to running mode.

Note

At least one another CAN node have to be connected to the CAN bus in order to synchronize the CAN communication.

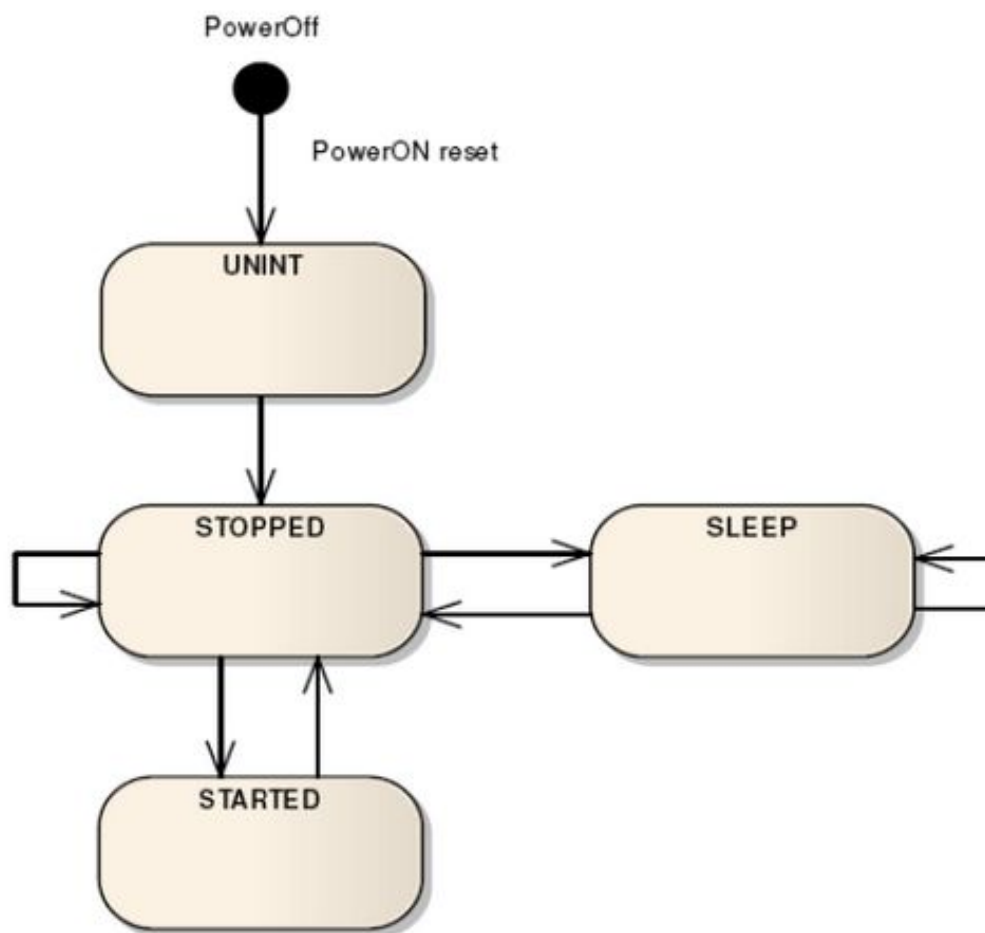


Figure 4-1. CAN Controller State Machine

4.1 Function Calls during Start-up

The CAN module shall be initialized by Can_Init() service call during the start-up. API service Can_SetController_Mode(Can_Controller, CAN_T_START) shall be used for setting the CAN controller to running mode.

Note

Pin settings are not related to Can driver or plugin configuration. GPIO pins used for connection of CAN physical layer have to be properly assigned to the FlexCAN module prior the CAN initialization.

4.2 Function Calls during Shutdown

The FlexCAN IP has many Low Power Modes, with programmable wake up on bus activity.

- **Freeze Mode**

This low power mode is entered when the HALT and FRZ bits in the MCR Register are asserted.

Module ignores the Rx input pin and drives the Tx pin as recessive, stops the prescaler, thus halting all CAN protocol activities and grants write access to the Error Counters Register (ECR), which is read-only in other modes.

Exit from this mode is done by negating the FRZ and HALT bits in the MCR Register or when the MCU is removed from Debug Mode

Note

It is not possible to exit from this mode by receiving a message on the Can bus.

- **Module Disable Mode**

This low power mode is entered when the MDIS bit in the MCR Register is asserted.

Module shuts down the clocks to the CAN Protocol Interface and Message Buffer Management sub-modules.

Exit from this mode is done by negating the MDIS bit in the MCR Register.

Note

It is not possible to exit from this mode by receiving a message on the Can bus.

- **Stop Mode**

This low power mode is entered when Stop Mode is requested at MCU level.

When in Stop Mode, the module puts itself in an inactive state and then informs the CPU that the clocks can be shut down globally.

Exit from this mode happens when the Stop Mode request is removed or when activity is detected on the CAN bus and the Self Wake Up mechanism is enabled.

Note

Note that wake-up from Stop Mode only works when both bits, SLF_WAK and WAK_MSK, are asserted. If interrupt for Wakeup is implemented in INTC, the interrupt handler

can change the state of the controller to RUN mode if it is needed. When exit from this mode controller is usually in Freeze mode.

Note

Refer the Reference Manual if wakeup is supported by hardware (bits register are implemented and INTC has connected the CAN wakeup interrupt signal).

4.3 Function Calls during Wake-up

The controller can be wakeup by a message when it is in Stop mode only if interrupt for wakeup is enabled or self wakeup mechanism is enabled.

SLF_WAK	WAK_MSK	MCU Clocks Enabled	Wake-up Interrupt Generated
0	0	No	No
0	1	No	No
1	0	No	No
1	1	Yes	Yes

Figure 4-2. Wake-up from Stop Mode

CAN stack can be changed from SLEEP mode to STOP mode by calling the Can_SetControllerMode(CAN_T_WAKEUP) service call.

Note

Refer the Reference Manual if wakeup is supported by hardware (bits register are implemented and INTC has connected the CAN wakeup interrupt signal).

Chapter 5

Module Requirements

In the current implementation, CAN is using the services of Schedule Manager (Rte) for entering and exiting the critical regions. Rte implementation is done by the integrators of the MCAL using OS or non-OS services. For testing the CAN, stubs are used for SchM.

Some CAN driver global variables updates are performed by ISRs before calling the user notification functions. In order to avoid the scenario where an executing CAN driver function is preempted by a CAN ISR, which is modifying some of the global variables, some exclusive areas are defined.

The ISR critical regions must not block the other critical regions to avoid deadlocks. This is ensured by exiting the ISR critical region before calling the user notification functions.

The following critical regions are used in the CAN driver:

5.1 Exclusive Areas

CAN_EXCLUSIVE_AREA_00 - Used in “Can_DisableControllerInterrupts” function, to protect the variable for nesting level of enabling/disabling interrupts. Refer to CAN202 requirement.

CAN_EXCLUSIVE_AREA_01 - Used in “Can_EnableControllerInterrupts” function, to protect the variable for nesting level of enabling/disabling interrupts. Refer to CAN202 requirement.

CAN_EXCLUSIVE_AREA_02 - Used in “Can_Write” function to protect the operation for checking the status of MB and for reserving it as a free to use for transmission. If hardware transmit object is free the mutex for that HTH is set to “signaled”. Between this verification and signal operation the protection must be applied. Refer to CAN212 requirement.

CAN_EXCLUSIVE_AREA_03 - Used in “FlexCan_LLD_Process_Rx” function in Can_FlexCan_LLD.c, to protect the operation for checking the region we are in this moment, is a safe region or not.

CAN_EXCLUSIVE_AREA_04 - Used in “FlexCan_LLD_RxFifoFrameAvNotif” function in Can_FlexCan_LLD.c, to protect the operation for checking the region we are in this moment, is a safe region or not.

CAN_EXCLUSIVE_AREA_05 - Used in “Can_LLD_Cancel” function in Can_FlexCan_LLD.c, to protect the operation for checking the region we are in this moment, is a safe region or not.

CAN_EXCLUSIVE_AREA_06 - Used in “Can_LLD_Write” function in Can_FlexCan_LLD.c, to protect the operation for checking the region we are in this moment, is a safe region or not.

Critical Region Exclusive Matrix

Below is the table depicting the exclusivity between different critical region IDs from the Can driver. If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

The critical regions from interrupts are grouped in “Interrupt Service Routines Critical Regions (composed diagram)”. If an exclusive area is “exclusive” with the composed “Interrupt Service Routines Critical Regions (composed diagram)” group, it means that it is exclusive with each one of the ISR critical regions.

Table 5-1. Exclusive Areas

	CAN_EXCLUSIVE_AREA_00	CAN_EXCLUSIVE_AREA_01	CAN_EXCLUSIVE_AREA_02	CAN_EXCLUSIVE_AREA_03	CAN_EXCLUSIVE_AREA_04	CAN_EXCLUSIVE_AREA_05	CAN_EXCLUSIVE_AREA_06	Interrupt Service Routines Critical Regions (composed diagram)
CAN_EXCLUSIVE_AREA_00	X							X
CAN_EXCLUSIVE_AREA_01		X						X
CAN_EXCLUSIVE_AREA_02			X					X
CAN_EXCLUSIVE_AREA_03				X				X
CAN_EXCLUSIVE_AREA_04					X			X
CAN_EXCLUSIVE_AREA_05						X		X

Table continues on the next page...

Table 5-1. Exclusive Areas (continued)

	CAN_EXCL USIVE_AR EA_00	CAN_EXCL USIVE_AR EA_01	CAN_EXCL USIVE_AR EA_02	CAN_EXCL USIVE_AR EA_03	CAN_EXCL USIVE_AR EA_04	CAN_EXCL USIVE_AR EA_05	CAN_EXCL USIVE_AR EA_06	Interrupt Service Routines Critical Regions(c omposed diagram)
CAN_EXCL USIVE_ARE A_06							X	X
Interrupt Service Routines Critical Regions (composed diagram)	X	X	X	X	X	X	X	X

5.2 Peripheral Hardware Requirements

The CAN physical interface should be connected to the CAN module pins in order to get the CAN bus voltage levels.

There have to be another one CAN node present on the CAN bus in order to get the CAN bus synchronized.

5.3 ISR to Configure within OS - Dependencies

5.3.1 The following ISR's are used by the CAN driver

Table 5-2. CAN ISRs

ISR Name	Hardware Interrupt Vector
Can_IsrFCA_ERR	65
Can_IsrFCA_BO	66
Reserved	67
Can_IsrFCA_MB_00_03	68
Can_IsrFCA_MB_04_07	69
Can_IsrFCA_MB_08_11	70
Can_IsrFCA_MB_12_15	71

Table continues on the next page...

Table 5-2. CAN ISRs (continued)

ISR Name	Hardware Interrupt Vector
Can_IsrFCA_MB_16_31	72
Can_IsrFCA_MB_32_63	73
Can_IsrFCB_ERR	85
Can_IsrFCB_BO	86
Reserved	87
Can_IsrFCB_MB_00_03	88
Can_IsrFCB_MB_04_07	89
Can_IsrFCB_MB_08_11	90
Can_IsrFCB_MB_12_15	91
Can_IsrFCB_MB_16_31	92
Can_IsrFCB_MB_32_63	93
Can_IsrFCC_ERR	105
Can_IsrFCC_BO	106
Reserved	107
Can_IsrFCC_MB_00_03	108
Can_IsrFCC_MB_04_07	109
Can_IsrFCC_MB_08_11	110
Can_IsrFCC_MB_12_15	111
Can_IsrFCC_MB_16_31	112
Can_IsrFCC_MB_32_63	113
Can_IsrFCD_ERR	173
Can_IsrFCD_BO	174
Reserved	175
Can_IsrFCD_MB_00_03	176
Can_IsrFCD_MB_04_07	177
Can_IsrFCD_MB_08_11	178
Can_IsrFCD_MB_12_15	179
Can_IsrFCD_MB_16_31	180
Can_IsrFCD_MB_32_63	181
Can_IsrFCE_ERR	190
Can_IsrFCE_BO	191
Reserved	192
Can_IsrFCE_MB_00_03	193
Can_IsrFCE_MB_04_07	194
Can_IsrFCE_MB_08_11	195
Can_IsrFCE_MB_12_15	196
Can_IsrFCE_MB_16_31	197
Can_IsrFCE_MB_32_63	198

Table continues on the next page...

Table 5-2. CAN ISRs (continued)

ISR Name	Hardware Interrupt Vector
Can_IsrFCF_ERR	202
Can_IsrFCF_BO	203
Reserved	204
Can_IsrFCF_MB_00_03	205
Can_IsrFCF_MB_04_07	206
Can_IsrFCF_MB_08_11	207
Can_IsrFCF_MB_12_15	208
Can_IsrFCF_MB_16_31	209
Can_IsrFCF_MB_32_63	210

5.3.2 Macros for Interrupts

General Interrupts for every controller

CAN_BOISR(FC) expands ISR(Can_IsrFC##FC##_BO) for BusOff event.

CAN_WKPISR(FC) expands ISR(Can_IsrFC##FC##_WKP) for Wakeup event.

CAN_ERROR_NOTIFICATION_ENABLE == STD_ON

CAN_ERRISR(FC) expands ISR(Can_IsrFC##FC##_ERR) for error event.

CanCodeSizeOptimization = STD_ON: All related ISRs are routed to one ISR function.

CAN_MB_UNIISRS(FC) expands ISR(Can_IsrFC##FC##_UNI) for Rx and Tx MBs.

CAN_MB_UNITXISRS(FC) expands ISR(Can_IsrFC##FC##_UNI) for Tx MBs.

CAN_MB_UNIRXISRS expands ISR(Can_IsrFC##FC##_UNI) for Rx MBs.

CanCodeSizeOptimization = STD_OFF: All related ISRs have separate ISR functions taking care about the ISR processing.

CAN_RXFIFO_EVENTS(FC) expands ISR(Can_IsrFC##FC##_Rx FifoEvents) for all Fifo events.

CAN_MB_ISRS(FC, Name, IdMin, IdMax) expands ISR(Can_IsrFC##FC##_##Name) for a group of Rx or Tx MBs.

CAN_MB_RXISRS(FC, Name, IdMin, IdMax) expands ISR(Can_IsrFC##FC##_##Name) for a group of Rx MBs.

CAN_MB_TXISRS(FC, Name, IdMin, IdMax) expands
ISR(Can_IsrFC##FC##_##Name) for a group of Tx MBs.

Note

MPC560XB has the IFLAG1[7:4] bits assigned to the same interrupt, then the solution is to use CAN_RXFIFO_EVENTS interrupts macro generation. The CAN_RXFIFO_EVENT_UNIFIED define is generated by Tressos in Can_Cfg.h file and depends by the attribute “Can.CanConfigSet.RxFifoEventsUnified” from Resource properties file.

Interrupt Handlers Example for Can controller A:

ISR(Can_IsrFCA_BO)

ISR(Can_IsrFCA_WKP)

ISR(Can_IsrFCA_UNI)

ISR(Can_IsrFCA_RxFifoEvents) or ISR(Can_IsrFCA_FrameAv)

ISR(Can_IsrFCA_Overf) ISR(Can_IsrFCA_Warn)

ISR(Can_IsrFCA_MB_00_03)

ISR(Can_IsrFCA_MB_04_07)

ISR(Can_IsrFCA_MB_08_11)

ISR(Can_IsrFCA_MB_12_15)

ISR(Can_IsrFCA_MB_16_31)

ISR(Can_IsrFCA_MB_32_63)

Interrupt Handlers Example for Can controller B:

ISR(Can_IsrFCB_BO)

ISR(Can_IsrFCB_WKP)

ISR(Can_IsrFCB_UNI)

ISR(Can_IsrFCB_RxFifoEvents) or ISR(Can_IsrFCB_FrameAv)

ISR(Can_IsrFCB_Overf) ISR(Can_IsrFCB_Warn)

ISR(Can_IsrFCB_MB_00_03)

ISR(Can_IsrFCB_MB_04_07)

ISR(Can_IsrFCB_MB_08_11)

ISR(Can_IsrFCB_MB_12_15)

ISR(Can_IsrFCB_MB_16_31)

ISR(Can_IsrFCB_MB_32_63)

Interrupt Handlers Example for Can controller C:

ISR(Can_IsrFCC_BO)

ISR(Can_IsrFCC_WKP)

ISR(Can_IsrFCC_UNI)

ISR(Can_IsrFCC_RxFifoEvents) or ISR(Can_IsrFCC_FrameAv)

ISR(Can_IsrFCC_Overf) ISR(Can_IsrFCC_Warn)

ISR(Can_IsrFCC_MB_00_03)

ISR(Can_IsrFCC_MB_04_07)

ISR(Can_IsrFCC_MB_08_11)

ISR(Can_IsrFCC_MB_12_15)

ISR(Can_IsrFCC_MB_16_31)

ISR(Can_IsrFCC_MB_32_63)

Interrupt Handlers Example for Can controller D:

ISR(Can_IsrFCD_BO)

ISR(Can_IsrFCD_WKP)

ISR(Can_IsrFCD_UNI)

ISR(Can_IsrFCD_RxFifoEvents) or ISR(Can_IsrFCD_FrameAv)

ISR(Can_IsrFCD_Overf) ISR(Can_IsrFCD_Warn)

ISR(Can_IsrFCD_MB_00_03)

ISR(Can_IsrFCD_MB_04_07)

ISR(Can_IsrFCD_MB_08_11)

ISR(Can_IsrFCD_MB_12_15)

ISR(Can_IsrFCD_MB_16_31)

ISR(Can_IsrFCD_MB_32_63)

Interrupt Handlers Example for Can controller E:

ISR(Can_IsrFCE_BO)

ISR(Can_IsrFCE_WKP)

ISR(Can_IsrFCE_UNI)

ISR(Can_IsrFCE_RxFifoEvents) or ISR(Can_IsrFCE_FrameAv)

ISR(Can_IsrFCE_Overf) ISR(Can_IsrFCE_Warn)

ISR(Can_IsrFCE_MB_00_03)

ISR(Can_IsrFCE_MB_04_07)

ISR(Can_IsrFCE_MB_08_11)

ISR(Can_IsrFCE_MB_12_15)

ISR(Can_IsrFCE_MB_16_31)

ISR(Can_IsrFCE_MB_32_63)

Interrupt Handlers Example for Can controller F:

ISR(Can_IsrFCF_BO)

ISR(Can_IsrFCF_WKP)

ISR(Can_IsrFCF_UNI)

ISR(Can_IsrFCF_RxFifoEvents) or ISR(Can_IsrFCF_FrameAv)

ISR(Can_IsrFCF_Overf) ISR(Can_IsrFCF_Warn)

ISR(Can_IsrFCF_MB_00_03)

ISR(Can_IsrFCF_MB_04_07)

ISR(Can_IsrFCF_MB_08_11)

ISR(Can_IsrFCF_MB_12_15)

ISR(Can_IsrFCF_MB_16_31)

ISR(Can_IsrFCF_MB_32_63)

5.4 ISR Macro

MCAL drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions:

a. OS is not used - AUTOSAR_OS_NOT_USED is defined:

i. If USE_SW_VECTOR_MODE is defined:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, drivers' interrupt handlers are normal C functions and the prolog/epilog handle the context save and restore.

ii. If USE_SW_VECTOR_MODE is not defined:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, drivers' interrupt handlers must save and restore the execution context.

Freescal Semiconductor OS is used - AUTOSAR_OS_NOT_USED is not defined

```
#define ISR(IsrName) void OS_isr_##IsrName()
```

In this case, OS is handling the execution context when an interrupt occurs. Drivers' interrupt handlers are normal C functions.

Other vendor's OS is used - AUTOSAR_OS_NOT_USED is not defined. Please refer to the OS documentation for description of the ISR macro.

5.5 Other AUTOSAR Modules - Dependencies

- **Mcu:** This module shall be initialized before using CAN. This module is required for setting the system clock frequency (clock for CAN).
- **Det** (only if CanDevErrorDetect=true): This module is necessary for enabling Development error detection. The API function used is Det_ReportError(). The activation/deactivation of Development error detection is configurable using 'CanDevErrorDetect' configuration parameter.
- **EcuM:** This module is necessary for a reference to the Wakeup source for this controller as defined in the ECU State Manager.
- **Resource:** Sub-Derivative model is selected from Resource configuration.

Chapter 6

Main API Requirements

6.1 Main Functions Calls within BSW Scheduler

CAN Driver support 4 main functions that can be configured to be scheduled by BSW scheduler:

- FUNC (void, CAN_CODE) Can_MainFunction_Write(void)
- FUNC (void, CAN_CODE) Can_MainFuction_Read(void)
- FUNC (void, CAN_CODE) Can_MainFunction_Wakeup(void)
- FUNC (void, CAN_CODE) Can_MainFunction_BusOff(void)
- FUNC (void, CAN_CODE) Can_MainFunction_Mode(void)

These Autosar APIs are scheduled if these 4 events are configured to be in “Polling” mode by the following parameters:

- CanTxProcessing

```
#define CAN_TXPOLL_SUPPORTED (STD_ON)
```

- CanRxProcessing

```
#define CAN_RXPOLL_SUPPORTED (STD_ON)
```

- CanWakeupProcessing

```
#define CAN_BUSOFFPOLL_SUPPORTED (STD_ON)
```

- CanBusoffProcessing

```
#define CAN_WAKEUPPOLL_SUPPORTED (STD_ON)
```

The period for polling is configured by the following 5 parameters:

- CanMainFunctionWritePeriod

```
#define CAN_MAINFUNCTION_PERIOD_WRITE (uint32)0.0010U
```

- CanMainFunctionReadPeriod

```
#define CAN_MAINFUNCTION_PERIOD_READ (uint32)0.0010U
```

- CanMainFunctionWakeupPeriod

```
#define CAN_MAINFUNCTION_PERIOD_WAKEUP (uint32) 0.0010U
```

- CanMainFunctionBusoffPeriod

```
#define CAN_MAINFUNCTION_PERIOD_BUSOFF (uint32)0.0010U
```

- CanMainFunctionModePeriod

```
#define CAN_MAINFUNCTION_MODE_PERIOD (uint32)0.0010U
```

Note

A configuration for an hardware unit can be possible in such a way that one controller will handle events by interrupts and another by polling method.

6.2 API Requirements

Not Applicable.

6.3 Calls to Notification Functions, Callbacks, Callouts

Call-back Notifications

The CAN stack provides the following call-back notifications:

- **CanIf_TxConfirmation:** This CAN Interface call-back function is called when a CAN message has been transmitted.

```
FUNC (void, CAN_CODE) CanIf_TxConfirmation(PduIdType CanTxPduId)
```

- **CanIf_RxIndication:** This CAN Interface call-back function is called when valid CAN message is received.

```
FUNC (void, CAN_CODE) CanIf_RxIndication(uint8 Hrh, Can_IdType CanId, uint8 CanDlc,
uint8Ptr CanSduPtr)
```

- **CanIf_CancelTxConfirmation:** This CAN Interface call-back function is called when the CAN message has been canceled during the transmission.

```
FUNC (void, CAN_CODE) CanIf_CancelTxConfirmation(const Can_PduType * PduInfoPtr)
```

- **CanIf_ControllerBusOff:** This CAN Interface call-back function is called when the CAN controller reached the bus-off state (see CAN specification for further details).

```
FUNC (void, CAN_CODE) CanIf_ControllerBusOff(uint8 Controller)
```

User Notification

- None

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Post Build data:

```
#ifndef CAN_START_CONFIG_DATA_UNSPECIFIED
#undef CAN_START_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR

/*Memory Section for Post Build Data to be defined here. Example given in the next line*/

#pragma ghs section const=".pb CAN_cfg"
#endif

#ifndef CAN_STOP_CONFIG_DATA_UNSPECIFIED
#undef CAN_STOP_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR

/*End of section to be mentioned here. Example given in the next line.*/

#pragma ghs section
#endif
```

For Code:

```
#ifndef CAN_START_SEC_CODE
```

```
#undef CAN_START_SEC_CODE
#undef MEMMAP_ERROR
/*Memory Section for Code to be defined here.*/
#endif
```

```
#ifdef CAN_STOP_SEC_CODE
#undef CAN_STOP_SEC_CODE
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

For Variables:

```
#ifdef CAN_START_SEC_VAR_UNSPECIFIED
#undef CAN_START_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*Memory Section for Variables to be defined here.*/
#endif
```

```
#ifdef CAN_STOP_SEC_VAR_UNSPECIFIED
#undef CAN_STOP_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

For Constant data:

```
#ifdef CAN_START_SEC_CONST_UNSPECIFIED
```

```
#undef CAN_START_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*Memory Section for Constants to be defined here.*/
#endif

#ifdef CAN_STOP_SEC_CONST_UNSPECIFIED
#undef CAN_STOP_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*End of section to be mentioned here*/
#endif
```

7.2 Linker Command File

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8

Configuration Parameters

Configuration parameter class for Autosar CAN driver fall into the following variants as defined below:

8.1 Configuration Parameters

Specifies whether the configuration parameter shall be of configuration class Post Build

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
Can	IMPLEMENTATION_CONFIG_VARIANT	Pre Compile parameter for all Variants of Configuration	Pre compile
CanNonAutosar			
	CanDisableDemReportErrorStatus	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanApiErrorStatus	Pre Compile parameter for all Variants of Configuration	Pre compile
CanGeneral			
	CanDevErrorDetection	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanVersionInfoApi	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanIndex	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMainFunctionBusoffPeriod	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMainFunctionWakeupPeriod	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMultiplexedTransmission	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanHardwareCancellation	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanIdenticalIdCancellation	Pre Compile parameter for all Variants of Configuration	Pre compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	CanTimeoutDuration	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanLPduReceiveCalloutFunction	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanCounterRef	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMainFunctionReadPeriodRef	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMainFunctionWritePeriodRef	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanSupportTTCANRef	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanCrystalFrequencyHz	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanCodeSizeOptimization	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanExtendedIDSupport	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMBCountExtensionSupport	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanApiEnableMbAbort	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanEnableDualClockMode	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanChangeBaudrateApi	Pre Compile parameter for all Variants of Configuration	Pre compile
CanGeneral CanMainFunctionRWPeriods			
	CanMainFunctionReadPeriod	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanMainFunctionWritePeriod	Pre Compile parameter for all Variants of Configuration	Pre compile
CanController			
	CanHwChannel	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanControllerActivation	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanControllerBaseAddress	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanControllerId	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanRxProcessing	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanTxProcessing	Pre Compile parameter for all Variants of Configuration	Pre compile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	CanBusoffProcessing	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanWakeupProcessing	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanBccSupport	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanWakeupSupport	VariantPC or VariantPB	VariantPC or VariantPB
	CanListenOnlyMode	VariantPC or VariantPB	VariantPC or VariantPB
	CanLoopBackMode	VariantPC or VariantPB	VariantPC or VariantPB
	CanSoftwareBusOffRecovery	VariantPC or VariantPB	VariantPC or VariantPB
	CanAutoBusOffRecovery	VariantPC or VariantPB	VariantPC or VariantPB
	CanTrippleSamplingEnable	VariantPC or VariantPB	VariantPC or VariantPB
	CanWakeUpSourceFilter	VariantPC or VariantPB	VariantPC or VariantPB
	CanLowestBuffTransmitFirst	VariantPC or VariantPB	VariantPC or VariantPB
	CanLocalPriorityEn	VariantPC or VariantPB	VariantPC or VariantPB
	CanWarningEnable	VariantPC or VariantPB	VariantPC or VariantPB
	CanClockFromBus	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanCpuClockRef	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanCpuClockRef_Alternate	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanControllerRxFifoEnable	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanRxFifoWarningNotification	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanRxFifoOverflowNotification	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanErrorControllerNotifEn	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanErrorControllerNotification	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanWakeupSourceRef	Pre Compile parameter for all Variants of Configuration	Pre compile
	CanControllerDefaultBaudrate	Pre Compile parameter for all Variants of Configuration	Pre compile
CanController \CanControllerBaudRate			
	CanControllerBaudRate	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerCheckCanStand	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerPropSeg	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerSeg1	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerSeg2	VariantPC or VariantPB	VariantPC or VariantPB

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	CanSyncJumpWidth	VariantPC or VariantPB	VariantPC or VariantPB
	CanAdvancedSetting	VariantPC or VariantPB	VariantPC or VariantPB
	CanBusLength	VariantPC or VariantPB	VariantPC or VariantPB
	CanPropDelayOfTranceiver	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerPrescaler	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerPrescaler_Alt	VariantPC or VariantPB	VariantPC or VariantPB
CanController\CanRxFifo			
	CanControllerIDAcceptanceMode	VariantPC or VariantPB	VariantPC or VariantPB
	CanIDValue0	VariantPC or VariantPB	VariantPC or VariantPB
	CanIDValue1	VariantPC or VariantPB	VariantPC or VariantPB
	CanIDValue2	VariantPC or VariantPB	VariantPC or VariantPB
	CanIDValue3	VariantPC or VariantPB	VariantPC or VariantPB
	CanTableIDType	VariantPC or VariantPB	VariantPC or VariantPB
	CanMBFilterMaskValue	VariantPC or VariantPB	VariantPC or VariantPB
CanController\CanFilterMask			
	CanFilterMaskValue	VariantPC or VariantPB	VariantPC or VariantPB
CanController\CanHardwareObject			
	CanHandleType	VariantPC or VariantPB	VariantPC or VariantPB
	CanIdType	VariantPC or VariantPB	VariantPC or VariantPB
	CanIdValue	VariantPC or VariantPB	VariantPC or VariantPB
	CanMBPrio	VariantPC or VariantPB	VariantPC or VariantPB
	CanObjectId	VariantPC or VariantPB	VariantPC or VariantPB
	CanObjectType	VariantPC or VariantPB	VariantPC or VariantPB
	CanControllerRef	VariantPC or VariantPB	VariantPC or VariantPB
	CanFilterMaskRef	VariantPC or VariantPB	VariantPC or VariantPB
	CanMainFunctionRWPeriodRef	VariantPC or VariantPB	VariantPC or VariantPB
CanController\CanTTController			
	CanTTControllerApplWatchdogLimit	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerCycleCountMax	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerExpectedTxTrigger	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerExternalClockSynchronisation	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerGlobalTimeFiltering	VariantPC or VariantPB	VariantPC or VariantPB

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
	CanTTControllerInitialRefOffset	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerInterruptEnable	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerLevel2	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerNTUConfig	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerOperationMode	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerSyncDeviation	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerTURRestore	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerTimeMaster	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerTimeMasterPriority	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerTxEnableWindowLength	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerWatchTriggerGapTimeMark	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTControllerWatchTriggerTimeMark	VariantPC or VariantPB	VariantPC or VariantPB
	CanTTIRQProcessing	VariantPC or VariantPB	VariantPC or VariantPB

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating CAN:

1. Generate the required CAN configuration.
2. Allocate proper memory sections in MemMap.h and linker command file.
3. Make sure all include files for compilation are available.
4. Map the ISRs to their vector locations.
5. Compile and build the CAN module with all the dependent modules.



How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13CANASR4.0R1.0.1
Revision 1.2